

Lab 5 - Part 1: Automated Data Ingestion & Versioning

- **Course:** Engineering of Intelligent Models
- **Module:** M3. Model Orchestration & Automation
- **Focus:** Dynamic API Ingestion, Hydra Configuration, and DVC Versioning
- **Branch:** Lab5

1. Goal of the Laboratory

The objective of this first segment is to engineer a robust, reproducible data ingestion pipeline. In a production environment, data is never static. We require a mechanism to pull historical weather data dynamically without altering the underlying Python code.

By the end of this notebook, you will have:

1. Designed a hierarchical Hydra configuration to manage API endpoints, predefined geographical locations, and meteorological variables.
2. Implemented a Python module utilizing the official Open-Meteo client to retrieve historical data efficiently.
3. Versioned the resulting raw dataset using DVC to ensure cryptographic lineage for future model training.

2. Version Control: Switching Branches

Before proceeding with this Lab, ensure your repository is clean and branched.

Switch to a new branch for Lab 5.

```
In [ ]: # Commit your Lab 4 progress
!git add .
!git commit -m "Complete Lab 4: Apache Airflow initial orchestration."

# Create and switch to the Lab4 branch
!git checkout -b Lab5
```

3. Configuration Management (Hydra)

Hardcoding coordinates, dates, or variable names into your ingestion script creates "Configuration Debt". We will extract these parameters into a structured YAML file. This allows us to switch our data context (e.g., from Lisbon to Porto) purely via command-line overrides during Airflow orchestration.

Refactor the configuration file at `conf/api/openmeteo.yaml` to support new configurations.

```
api:
  endpoint: "https://archive-api.open-meteo.com/v1/archive"
  timezone: "auto"

# Predefined Locations dictionary. Allows easy swapping via Hydra
locations:
  Sintra:
    latitude: 38.801
    longitude: -9.3783
  Porto:
    latitude: 41.1496
    longitude: -8.6110
  Lisbon:
    latitude: 38.7167
    longitude: -9.1333

# Default Location selected for ingestion
```

```

target_location: "Sintra"

# Data ranges
date_range:
    start_date: "2026-02-06"
    end_date: "2026-02-20"

# Target variables as per Open-Meteo Historical API documentation
variables:
    - "temperature_2m"
    - "relative_humidity_2m"
    - "precipitation"

# Use Hydra interpolation to dynamically inject the location name into the filename
output:
    raw_data_path: "data/raw/historical_weather-${target_location}.csv"

```

4. The Ingestion Implementation

We will use the `openmeteo-requests`, `requests-cache`, and `retry-requests` libraries (already included in your `requirements.txt` from Lab 4) to ensure our API calls are resilient to network failures and rate limits.

Create the a new ingestion script at `src/ingestion/get_historical_data.py` (we're preserving the older `get_data.py` for now).

```

import hydra
from omegaconf import DictConfig
from typing import Dict, Any
import openmeteo_requests
import requests_cache
import pandas as pd
from retry_requests import retry
import logging
import os

# Configure basic logging
logging.basicConfig(level=logging.INFO, format='[%asctime)s] %(levelname)s - %
(message)s')
logger = logging.getLogger(__name__)

@hydra.main(version_base=None, config_path="../../conf", config_name="config")
def ingest_data(cfg: DictConfig) -> None:
    """
    Retrieves historical weather data from Open-Meteo API based on Hydra configuration
    and saves it to the specified raw data path.
    """

    logger.info("--- Starting Data Ingestion Pipeline ---")

    # 1. Setup the Open-Meteo API client with cache and retry on error
    cache_session = requests_cache.CachedSession('.cache', expire_after=-1)
    retry_session = retry(cache_session, retries=5, backoff_factor=0.2)
    openmeteo = openmeteo_requests.Client(session=retry_session)

    # 2. Extract configurations
    location_name = cfg.api.target_location
    coords = cfg.api.locations[location_name]
    dates = cfg.api.date_range
    variables = cfg.api.variables
    output_path = cfg.api.output.raw_data_path

    logger.info(f"Target Location: {location_name} (Lat: {coords.latitude}, Lon:
{coords.longitude})")

```

```

logger.info(f"Date Range: {dates.start_date} to {dates.end_date}")

# 3. Construct the API Payload
params = {
    "latitude": coords.latitude,
    "longitude": coords.longitude,
    "start_date": dates.start_date,
    "end_date": dates.end_date,
    "hourly": list(variables),
    "timezone": cfg.api.api.timezone
}

# 4. Execute the API Request
logger.info("Fetching data from Open-Meteo Historical API...")
url = cfg.api.api.endpoint
responses = openmeteo.weather_api(url, params=params)

# Process the first Location (we only requested one)
response = responses[0]
hourly = response.Hourly()

# 5. Process Data into a Pandas DataFrame
# Note: Constructing the time index correctly to match the array lengths
start_time = pd.to_datetime(hourly.Time(), unit="s", utc=True)
end_time = pd.to_datetime(hourly.TimeEnd(), unit="s", utc=True)
interval = pd.Timedelta(seconds=hourly.Interval())

# Create the date range - this ensures the index length matches the variables
date_range = pd.date_range(
    start=start_time,
    end=end_time,
    freq=interval,
    inclusive="left"
)

# Initialize the dictionary with the date and the Location name
hourly_data: Dict[str, Any] = {
    "date": date_range,
    "location": [location_name] * len(date_range) # Broadcast Location name to all
rows
}

# Dynamically map the requested variables to the response arrays
for idx, var_name in enumerate(variables):
    hourly_data[var_name] = hourly.Variables(idx).ValuesAsNumpy()

new_df = pd.DataFrame(data=hourly_data)

# 6. Incremental Loading & Deduplication
os.makedirs(os.path.dirname(output_path), exist_ok=True)

if os.path.exists(output_path):
    logger.info(f"Existing dataset found at {output_path}. Appending new data...")
    # Read existing data
    existing_df = pd.read_csv(output_path)

    # Convert date strings back to datetime objects for accurate comparison
    existing_df['date'] = pd.to_datetime(existing_df['date'], utc=True)

    # Concatenate old and new data
    combined_df = pd.concat([existing_df, new_df], ignore_index=True)

    # Drop duplicates based on exact date and Location.
    # keep='last' ensures that if we fetch newer data for the same date, it
overwrites the old.
    combined_df = combined_df.drop_duplicates(subset=['date', 'location'],

```

```

keep='last')

    # Sort chronologically
    combined_df = combined_df.sort_values(by=['date']).reset_index(drop=True)

    new_records_count = len(combined_df) - len(existing_df)
    logger.info(f"Added {new_records_count} new unique records.")
    final_df = combined_df
else:
    logger.info(f"No existing dataset found. Creating new file at {output_path}.")
    final_df = new_df

# Format the dates cleanly for the CSV
final_df['date'] = final_df['date'].dt.strftime('%Y-%m-%d %H:%M:%S')

# Save to CSV
final_df.to_csv(output_path, index=False)

logger.info(f"Successfully ingested {len(final_df)} records.")
logger.info(f>Data saved to {output_path}")
logger.info("--- Data Ingestion Complete ---")

if __name__ == "__main__":
    ingest_data()

```

Code Explanation

The code above is very similar to the previous `get_data.py` script, but with several critical enhancements:

- `params` : The API parameters are now dynamically constructed from the Hydra configuration, allowing for flexible location and variable selection without code changes.
- `output_path` : The output CSV path is also dynamically generated using Hydra interpolation, ensuring that different locations produce separate files.
- `if os.path.exists(output_path)` : This block implements incremental loading. If a CSV already exists, it reads the existing data, concatenates it with the new data, and drops duplicates based on the combination of `date` and `location` . This ensures that if we fetch overlapping date ranges in the future, we won't create duplicate records.
- `logger` : We use Python's built-in logging module to provide clear, timestamped logs of the ingestion process, which is crucial for debugging and monitoring in production environments.

5. Data Lineage and Versioning (DVC)

Once the script is executed and the CSV is generated, we must freeze this specific state of the data. Machine Learning reproducibility dictates that we must always know exactly which data was used to train a specific model.

Step 1: Execute the script locally to generate the initial dataset.

```

In [ ]: # Change the root directory for the script to work (if you're running my repo dir structure)
import os
os.chdir("../..")

!python src/ingestion/get_historical_data.py

```

Step 2: Tell DVC to update its snapshot of the entire folder

```

In [ ]: !dvc add data/raw

```

Step 3: Make sure that `raw.dvc` is on git to allow DVC folder tracking.

```
In [ ]: # DVC will automatically update the data/raw.dvc file with a new hash.
!git add data/raw.dvc
```

6. Overriding Configurations via CLI

To prove the resilience of your architecture, you can now ingest data for a completely different location without changing a single line of Python code.

If you wanted to retrieve data for Porto, you would simply execute:

```
In [ ]: !python src/ingestion/get_historical_data.py api.target_location=Porto
```

If everything ran smoothly, you should see a new file under `data/raw` named `historical_weather-Porto.csv`.

7. Orchestrating the Ingestion Pipeline (Apache Airflow)

Now that our Python script is capable of incremental loading and dynamic configuration via Hydra, we must refactor our Airflow orchestration.

In a production MLOps environment, Data Engineering (Ingestion) and Data Science (Training) have different lifecycles and failure states. For example, we want to fetch new weather data every day, but only retrain our forecasting models once a month. To achieve this **Separation of Concerns**, we must split our monolithic pipeline into two distinct, decoupled DAGs.

Furthermore, we will implement **Parameterized DAGs**. Airflow allows us to define UI forms where users can input specific variables (like date ranges) right before manually triggering a run. We will use Jinja templating (`{{ params.variable }}`) to pass these UI inputs directly into our Python script via Hydra CLI overrides.

Step 1: Delete the old DAG

Remove the old `dags/weather_pipeline.py` file to avoid confusion.

Step 2: Create the Daily Ingestion DAG

Create a new file at `dags/data_ingestion_dag.py` . Notice the `params` dictionary defining the UI form, and how those parameters are injected into the `BashOperator` to override the Hydra configuration.

```
# dags/data_ingestion_dag.py
from airflow.sdk import DAG
from airflow.sdk import Param
from airflow.sdk.definitions.param import ParamsDict
from airflow.sdk.definitions.dag import DAG
from airflow.providers.standard.operators.bash import BashOperator
from datetime import datetime, timedelta

# Default arguments applied to all tasks in the DAG
default_args = {
    'owner': 'mlops_engineer',
    'depends_on_past': False,
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

# Define the DAG with UI Parameters
with DAG(
    'daily_weather_ingestion',
    default_args=default_args,
    description='Fetches historical weather data incrementally',
    schedule='@daily', # Runs once a day automatically
```

```

catchup=False,
tags=['weather_capstone', 'ingestion'],
params=ParamsDict({
    "start_date": Param("2026-02-01", type="string", format="date",
description="Start date (YYYY-MM-DD)" ),
    "end_date": Param("2026-02-28", type="string", format="date", description="End
date (YYYY-MM-DD)" ),
})
) as dag:
    # Task 1: Ingest Data for Lisbon
    ingest_lisbon = BashOperator(
        task_id='ingest_weather_lisbon',
        bash_command='cd /opt/airflow && python src/ingestion/get_historical_data.py '
                    'api.target_location="Lisbon" '
                    'api.date_range.start_date="{{ params.start_date }}" '
                    'api.date_range.end_date="{{ params.end_date }}"'
    )

    # Task 2: Ingest Data for Porto (Runs in parallel with Lisbon)
    ingest_porto = BashOperator(
        task_id='ingest_weather_porto',
        bash_command='cd /opt/airflow && python src/ingestion/get_historical_data.py '
                    'api.target_location="Porto" '
                    'api.date_range.start_date="{{ params.start_date }}" '
                    'api.date_range.end_date="{{ params.end_date }}"'
    )

    # Task 3: Update DVC Tracking
    update_dvc = BashOperator(
        task_id='update_dvc_tracking',
        bash_command='cd /opt/airflow && dvc add data/raw'
    )

    # Define the execution flow
    [ingest_lisbon, ingest_porto] >> update_dvc

```

Code Explanation

This DAG is also similar to the previous one, but with several critical enhancements due to the separation of concerns and parameterization feature of Airflow:

- **params** : We define a `ParamsDict` at the DAG level, which creates a UI form for users to input `start_date` and `end_date` when manually triggering the DAG. You can also set default values for these parameters, which will be used if the user does not override them.
 - In this lab, I've only included the parameters for the date range, but you could easily extend this to include the target location as well, allowing for even more flexible runs.
- **Task 2:** `ingest_lisbon` and `ingest_porto` : Both tasks now use Jinja templating (`{{ params.variable }}`) to inject the user-provided parameters directly into the command that runs the Python script.
 - Also, notice that we are using Hydra CLI overrides to pass the parameters, which means we don't need to hardcode any logic in our Python script to handle Airflow-specific parameters. The Python script remains completely agnostic of the orchestration layer, adhering to the principle of separation of concerns.
- **Task 3:** `update_dvc` : Now, after both ingestion tasks complete, we run a DVC command to update the tracking of the `data/raw` folder.
- `[ingest_lisbon, ingest_porto] >> update_dvc` : This line defines the execution flow. Both ingestion tasks run in parallel, and only after both have completed successfully does the DVC update task execute. This ensures that we only update our data versioning after all new data has been ingested.

Step 3. Testing the New DAG via the Airflow UI

To verify that our incremental loading, parameter injection, and DVC tracking work together seamlessly, we will execute a manual test run using custom dates.

1. **Access the UI:** Open your browser and navigate to `http://localhost:8080`.
2. **Trigger with Config:** Locate the `daily_weather_ingestion` DAG. Click on the **Trigger** button (the play icon) and select the **Single Run** option from the menu.

The screenshot shows the MLflow UI interface for triggering a DAG run. The 'Trigger Dag - daily_weather_ingestion' modal is open, displaying the 'Single Run' configuration. The 'Run Parameters' section is visible, showing 'start_date' as '01/02/2026' and 'end_date' as '28/02/2026'. The 'Trigger' button is at the bottom right. On the right side of the modal, a table shows the DAG run status as 'Success'.

Run	Start Date	End Date	Duration
52.991251+00:00	2026-03-12 10:03:53	2026-03-12 10:04:01	00:00:07.058

3. **Inject Parameters:** You will be presented with the parameter form. Change the `start_date` to `2026-01-15` and `end_date` to `2026-01-31`. Confirm and trigger the run.
4. **Monitor Execution:** Click on the DAG name to view the execution. You should see the Lisbon and Porto ingestion tasks running in parallel, successfully followed by the DVC update task.

The screenshot shows the MLflow UI interface for monitoring a DAG run. The DAG 'daily_weather_ingestion' is shown with three tasks: 'ingest_weather_porto', 'ingest_weather_lisbon', and 'update_dvc_tracking', all marked as 'success'. The right panel shows the 'Task Instances' table with details for each task.

Task ID	Map Index	State	Start Date
update_dvc_tracking		Success	2026-03-12 10:03:58
ingest_weather_porto		Success	2026-03-12 10:03:54
ingest_weather_lisbon		Success	2026-03-12 10:03:54

5. **Verify Outputs:** After the run completes, check the `data/raw` folder. You should see new CSV files for both Lisbon and Porto with data covering the specified date range. Additionally, check that the `raw.dvc` file has been updated with new hashes.

By completing this notebook, you have built a fault-tolerant, idempotent, and highly configurable Data Engineering pipeline.

8. Next Steps

In the next notebook (Notebook 2), we will transition from Data Engineering to Data Science. We will utilize this automatically versioned data to design PyTorch Lightning (LSTM/GRU) and Prophet architectures, managing their hyperparameters with Hydra and tracking their loss curves directly in MLflow.